



Machine Learning Lab: Optimizing Neural Networks on Medical Imaging

Hands-on Engineering Sprint: Overcoming Class Imbalance and Architectural Limits on BreastMNIST

Course: Applied Deep Learning & Neural Network Architecture

Objective: Optimize Baseline Validation Accuracy & Macro F1-Score

Dataset: BreastMNIST (28x28 Grayscale, 2 Classes)

Framework: PyTorch 2.x & Torchvision

1. Lab Overview & Motivation

In standard machine learning tutorials, boilerplate scripts often achieve deceptively high accuracy scores by exploiting dataset quirks or majority-class skew. In real-world biomedical engineering—such as diagnosing breast cancer from ultrasound or pathology slices—blindly chasing overall accuracy is a critical failure mode.

In this lab, you will take a naive multi-layer perceptron (MLP) trained on **BreastMNIST** and act as an ML optimization engineer. Your mission is to diagnose why the baseline plateaus, tackle class imbalance, upgrade the architecture, and rigorously improve **two conflicting metrics** simultaneously: **Accuracy** and **Macro F1-Score** (or AUPRC).

The Core Engineering Challenge

A model that predicts the majority class 100% of the time might achieve high baseline accuracy, but its Macro F1-score will collapse toward zero. Your goal is to break this trade-off through intelligent loss engineering, regularization, and feature extraction.

2. The Baseline Implementation

Below is your starting boilerplate code. It loads the **BreastMNIST** dataset, wraps it in PyTorch **DataLoader** objects, builds a flat 3-layer neural network, and runs a standard training and evaluation loop.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision.transforms import v2
import medmnist
from medmnist import BreastMNIST

# 1. Data preprocessing and augmentation pipeline
data_transform = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=[0.5], std=[0.5])
])

# Download and load BreastMNIST datasets
training_data = BreastMNIST(split="train", transform=data_transform, download=True)
```

```

test_data = BreastMNIST(split="test", transform=data_transform, download=True)

batch_size = 64
train_dataloader = DataLoader(training_data, batch_size=batch_size, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=batch_size, shuffle=False)

device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
print(f"Using {device} device")

# 2. Define baseline model (MLP)
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 2) # BreastMNIST has 2 classes (malignant/benign)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

def train(dataloader, model, loss_fn, optimizer):
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)
        y = y.squeeze().long() # Essential: MedMNIST labels are [N, 1]

        pred = model(X)
        loss = loss_fn(pred, y)

        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

def test(dataloader, model, loss_fn):
    model.eval()
    test_loss, correct, total = 0, 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            y = y.squeeze().long()
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()
            total += y.size(0)
    print(f"Test Accuracy: {(100*correct/total):>0.1f}%, Avg Loss: {test_loss/len(dataloader):>8f}")

```

```
for t in range(5):
    print(f"Epoch {t+1}\n-----")
    train(train_dataloader, model, loss_fn, optimizer)
    test(test_dataloader, model, loss_fn)
print("Done!")
```

3. Assignment Requirements & Objectives

You must fork or copy the baseline script and implement improvements to maximize performance across two required metrics. Your final submission will be graded on your experimental methodology and empirical gains.

Metric Category	Target Metric	Baseline Problem	Required Action
 Primary Performance	Overall Accuracy	Underfitting due to linear bottleneck in flat MLP layers.	Upgrade architecture to a Convolutional Neural Network (CNN) or Residual Block.
 Robustness & Fairness	Macro F1-Score	Class skew causes the model to ignore minority malignant classes.	Implement weighted CrossEntropyLoss, Focal Loss, or advanced data augmentation.

4. Suggested Engineering Levers

You are free to experiment with any combination of the following techniques. Document your hypotheses before testing each lever:

-  **Architectural Evolution:** Replace the flat `nn.Linear` stack with a custom 2D Convolutional Neural Network (Conv2D -> BatchNorm2d -> ReLU -> MaxPool2d) to capture local spatial structures in ultrasound images.
-  **Loss Engineering:** Compute class frequencies in the training set, derive inverse class weights, and pass them into `nn.CrossEntropyLoss(weight=class_weights)` to penalize minority class misclassifications.
-  **Data Augmentation:** Integrate torchvision transforms such as `v2.RandomHorizontalFlip()`, `v2.RandomRotation(15)`, or `v2.ColorJitter()` to improve model generalizability.
-  **Optimizer & Scheduler Tuning:** Experiment with AdamW, learning rate warmups, or `ReduceLROnPlateau` schedulers to escape local minima.

Deliverables & Evaluation

Submit a Python script (.py or Jupyter Notebook) containing your optimized pipeline, along with a 2-page report detailing your baseline metrics, your experimental iterations, and final comparative results for Accuracy and Macro F1-Score.